

Security Audit

Arcane 2nd (DeFi)

Table of Contents

Executive Summary	4
Project Context	4
Audit Scope	7
Security Rating	8
Intended Program Functions	10
Code Quality	11
Audit Resources	11
Dependencies	11
Severity Definitions	12
Status Definitions	13
Audit Findings	14
Centralisation	18
Conclusion	19
Our Methodology	20
Disclaimers	22
About Hashlock	23

CAUTION

THIS DOCUMENT IS A SECURITY AUDIT REPORT AND MAY CONTAIN CONFIDENTIAL INFORMATION. THIS INCLUDES IDENTIFIED VULNERABILITIES AND MALICIOUS CODE WHICH COULD BE USED TO COMPROMISE THE PROJECT. THIS DOCUMENT SHOULD ONLY BE FOR INTERNAL USE UNTIL ISSUES ARE RESOLVED. ONCE VULNERABILITIES ARE REMEDIATED, THIS REPORT CAN BE MADE PUBLIC. THE CONTENT OF THIS REPORT IS OWNED BY HASHLOCK PTY LTD FOR USE OF THE CLIENT.

Executive Summary

The Arcane team partnered with Hashlock to conduct a security audit of their code. Hashlock manually and proactively reviewed the code in order to ensure the project's team and community that the deployed contracts are secure.

Project Context

Arcane is a Web3 privacy-focused infrastructure project building confidential and compliance-oriented blockchain solutions for decentralized finance and cross-chain applications. The protocol focuses on enabling private transactions, shielded asset transfers, and privacy-preserving DeFi interactions through advanced cryptographic mechanisms and decentralized infrastructure. Arcane's ecosystem includes components such as private liquidity infrastructure, privacy-enhanced wallets, cross-chain interoperability mechanisms, and institutional-grade compliance tooling designed to support secure and confidential on-chain activity across multiple blockchain ecosystems.

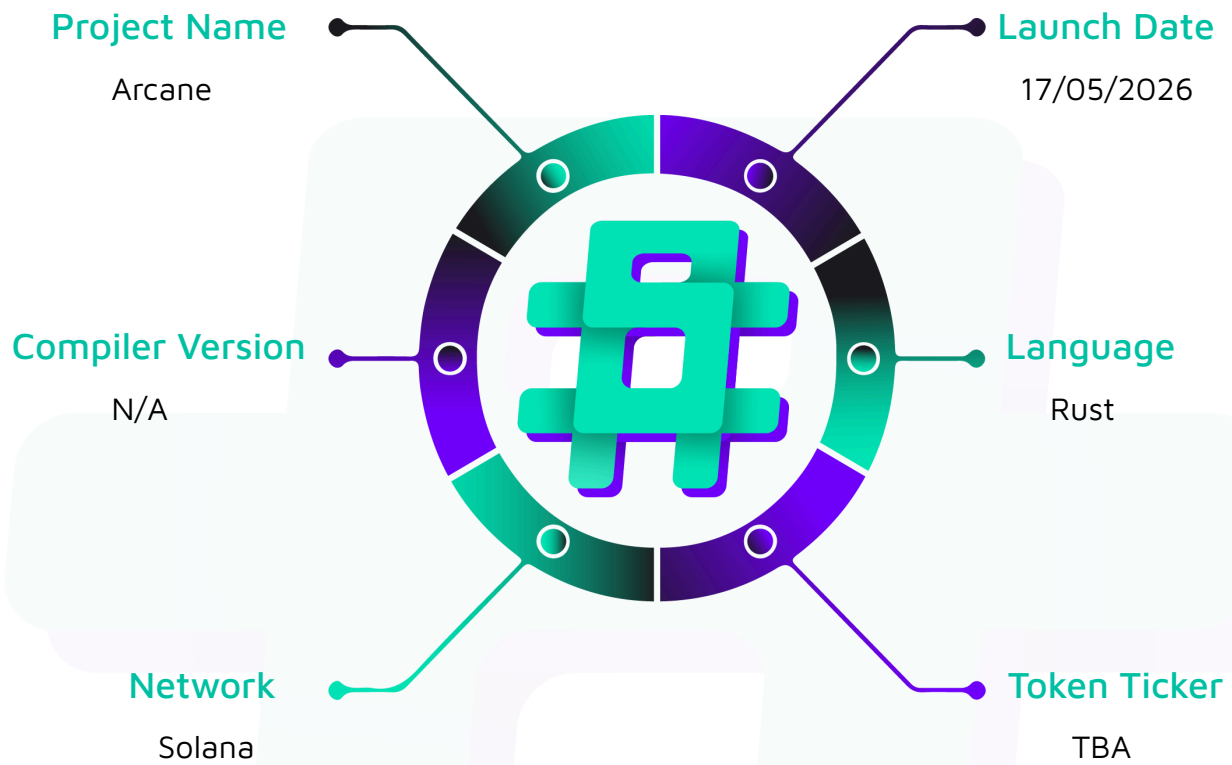
Project Name: Arcane

Project Type: DeFi

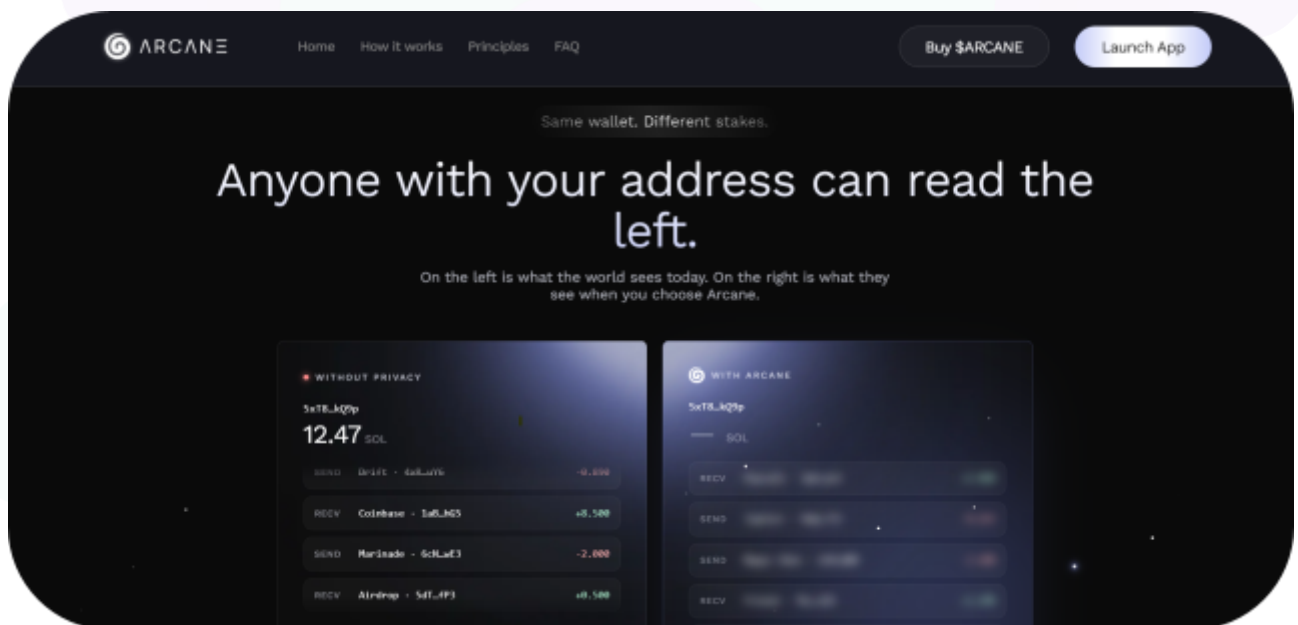
Website: <https://arcaneprivacy.com/>

Logo:



Visualised Context:

Project Visuals:



Audit Scope

We at Hashlock audited the Rust code within the Arcane project, the scope of work included a comprehensive review of the programs and modules listed below. We tested the programs to assess their security, functionality, and efficiency. These tests were undertaken primarily through manual line-by-line analysis and were supported by software-assisted testing.

Description	Arcane Smart Contracts
Platform	Solana / Rust
Audit Date	July, 2026
Contract 1	add_pool.rs
Contract 2	backup_notes.rs
Contract 3	common.rs
Contract 4	deposit.rs
Contract 5	init_network.rs
Contract 6	nit_notes.rs
Contract 7	mod.rs
Contract 8	update_network.rs
Contract 9	withdraw.rs
Contract 10	commitment.rs
Contract 11	mod.rs
Contract 12	network_state.rs
Contract 13	notes.rs
Contract 14	nullifier.rs
Contract 15	pool.rs
Contract 16	poseidon_merkle_tree.rs
Contract 17	error.rs

Contract 18	events.rs
Contract 19	lib.rs
Contract 20	verifying_key.rs
Audited GitHub Commit Hash	8ca8591ff09a07f8788c8d82411dcdf4232328bf
Fix Review GitHub Commit Hash	4cf34c52017097a5da421ccd5250d8b0a4f6948b

Note: Hashlock initially audited the code associated with the "Audited GitHub Commit Hash" and the findings presented in this report pertain specifically to that commit. For the "Fix Review GitHub Commit Hash", Hashlock solely verified the status of the identified findings and did not conduct a comprehensive review to assess any additional code implementations.

Security Rating

After Hashlock's Audit, we found the codebase to be **"Secure"**. The code all follows simple logic, with correct and detailed ordering. They use a series of interfaces and external dependencies, including standard open-source libraries.



The 'Hashlocked' rating is reserved for projects that ensure ongoing security via bug bounty programs or on chain monitoring technology.

All issues uncovered during automated and manual analysis were meticulously reviewed and applicable vulnerabilities are presented in the [Audit Findings](#) section. The list of audited assets is presented in the [Audit Scope](#) section and the project's contract functionality is presented in the [Intended Program Functions](#) section.

All vulnerabilities initially identified have now been resolved and acknowledged.

Hashlock found:

1 Low severity vulnerability

3 QA

Caution: *Hashlock's audits do not guarantee a project's success or ethics, and are not liable or responsible for security. Always conduct independent research about any project before interacting.*

Intended Program Functions

Claimed Behaviour	Actual Behaviour
lib.rs Program entrypoint. Defines the instructions and their account-validation contexts, token pool creation, token deposits and withdrawals, and the NotesV2 backup and dispatches each to its processor handler.	Contract achieves this functionality.
processor/deposit.rs Deposit flow for SOL and SPL/Token-2022. Screens the depositor against the Range risk oracle, records the commitment in the pool's Merkle tree, and moves funds into the treasury.	Contract achieves this functionality.
processor/withdraw.rs Withdrawal flow for SOL and SPL. Verifies the Groth16 proof against a known Merkle root, blocks double-spends via the nullifier, and pays the recipient, the relayer fee, and the protocol fee split.	Contract achieves this functionality.
processor/add_pool.rs Creates fixed-denomination SOL and token pools, each with its own Merkle tree.	Contract achieves this functionality.
processor/common.rs Builds and validates the network configuration (authority, fee wallets, fee splits) and provides the oracle risk-score verification helper used by deposits.	Contract achieves this functionality.
processor/backup_notes.rs Stores the user's encrypted notes on-chain for recovery.	Contract achieves this functionality.

state/notes.rs Defines the NotesV2 account that holds a user's encrypted note entries.	Contract achieves this functionality.
--	--



Code Quality

This audit scope involves the smart contracts of the Arcane project, as outlined in the Audit Scope section. All contracts, libraries, and interfaces mostly follow standard best practices and to help avoid unnecessary complexity that increases the likelihood of exploitation, however, some refactoring were recommended to optimize security measures.

The code is very well commented on and closely follows best practice nat-spec styling. All comments are correctly aligned with code functionality.

Audit Resources

We were given the Arcane project smart contract code in the form of Github access.

As mentioned above, code parts are well commented. The logic is straightforward, and therefore it is easy to quickly comprehend the programming flow as well as the complex code logic. The comments are helpful in providing an understanding of the protocol's overall architecture.

Dependencies

As per our observation, the libraries used in this smart contracts infrastructure are based on well-known industry standard open source projects.

Apart from libraries, its functions are used in external smart contract calls.

Severity Definitions

The severity levels assigned to findings represent a comprehensive evaluation of both their potential impact and the likelihood of occurrence within the system. These categorizations are established based on Hashlock's professional standards and expertise, incorporating both industry best practices and our discretion as security auditors. This ensures a tailored assessment that reflects the specific context and risk profile of each finding.

Significance	Description
High	High-severity vulnerabilities can result in loss of funds, asset loss, access denial, and other critical issues that will result in the direct loss of funds and control by the owners and community.
Medium	Medium-level difficulties should be solved before deployment, but won't result in loss of funds.
Low	Low-level vulnerabilities are areas that lack best practices that may cause small complications in the future.
Gas	Gas Optimisations, issues, and inefficiencies.
QA	Quality Assurance (QA) findings are informational and don't impact functionality. Supports clients improve the clarity, maintainability, or overall structure of the code.

Status Definitions

Each identified security finding is assigned a status that reflects its current stage of remediation or acknowledgment. The status provides clarity on the handling of the issue and ensures transparency in the auditing process. The statuses are as follows:

Significance	Description
Resolved	The identified vulnerability has been fully mitigated either through the implementation of the recommended solution proposed by Hashlock or through an alternative client-provided solution that demonstrably addresses the issue.
Acknowledged	The client has formally recognized the vulnerability but has chosen not to address it due to the high cost or complexity of remediation. This status is acceptable for medium and low-severity findings after internal review and agreement. However, all high-severity findings must be resolved without exception.
Unresolved	The finding remains neither remediated nor formally acknowledged by the client, leaving the vulnerability unaddressed.

Audit Findings

Low

[L-01] arcane#add_token_pool - Token pools accept any token without checking its properties

Description

When the operator adds a new token pool, the program accepts any token mint without checking what kind of token it is. Most tokens are fine, but a few standard token features can leave a pool unable to pay everyone back, or let the token issuer freeze the pooled funds. Because all deposits for a token sit together in one shared vault, a problem with one listed token affects every depositor of that token, not just one person.

Vulnerability Details

`add_token_pool` takes the mint with no constraints, and the only check is on the amount:

```
pub mint: InterfaceAccount<'info, Mint>,      // no checks on the token
require!(denomination > 0, Error::InvalidDenomination);
```

Two token features are the practical concern:

- ****Fee-on-transfer tokens.**** Some tokens charge a fee on every transfer. The pool then receives slightly less than the full amount on each deposit, but still pays out the full amount on withdrawal. Over many deposits the vault runs short, and the last people to withdraw cannot be paid.
- ****Freeze / seize authority (e.g. USDC, USDT).**** The token issuer can freeze the pool's token account, locking everyone's funds, or move them under a legal order.

Two more token types simply make a pool unusable or, in a bad configuration, drainable: a ****transfer-hook**** token makes every deposit and withdrawal fail (the program does not pass the extra accounts such tokens require), and a

****permanent-delegate**** token can be moved by whoever holds that authority. These require an untrustworthy or misconfigured token to be listed in the first place.

Impact

A pool for the wrong kind of token can become insolvent (some depositors cannot withdraw) or be frozen by the issuer. The effect is limited to that one token's pools; other tokens and the SOL pools are unaffected. Rated Low because it only happens if such a token is listed, and listing is controlled by the trusted operator.

Recommendation

Check tokens before listing them. Either keep a small allowlist of approved token mints, or reject tokens that have a transfer fee, transfer hook, permanent delegate, or (where relevant) an issuer freeze authority. This is standard practice on Solana — for example, Orca uses an allowlist ("TokenBadge") for exactly this reason.

Status

Acknowledged

QA

[Q-01] arcane#handle_backup_notes - No explicit size cap on encrypted notes

Description

The v0.2.0 note-backup account stores arbitrary-length binary notes with no per-note or total-size cap, so a caller can grow the account up to the runtime limits at their own rent expense.

Vulnerability Details

NotesV2 replaced the fixed 252-byte String model with unbounded binary entries:

```
#[account]
pub struct NotesV2 {
    pub entries: Vec<Vec<u8>>, // each note is arbitrary binary data
}
```

handle_backup_notes appends whatever is submitted, with no length validation:

```
notes.entries.extend(encrypted_notes);
```

Growth is bounded only by the Solana transaction-size limit (~1232 bytes/tx) and the per-instruction realloc increment (~10 KB), and the caller funds the rent for the account growth.

Impact

No security impact; self-limited. The submitter pays their own rent and can only grow their own notes account; there is no effect on pooled funds or other users. It is a resource-hygiene concern, not an exploit.

Recommendation

Add an explicit per-note length cap and a total-entries (or total-bytes) cap in handle_backup_notes, and reject submissions that exceed them.

Status

Acknowledged

[Q-02] arcane#handle_token_withdraw / handle_withdraw - Risk screening runs on deposit only, not on withdrawal

Description

The v0.2.0 code removes the Range risk/sanctions screening from the withdrawal path, so on-chain screening now runs on deposit only. This appears intentional, but it diverges from the public Compliance documentation, which still guarantees screening of withdrawals.

Vulnerability Details

In the audited baseline, `handle_withdraw` screened the `recipient` against the Range risk/sanctions oracle via `verify_risk_score_feed`. The current code removes that call from **both** `handle_withdraw` and `handle_token_withdraw`, and the `Withdraw` context no longer carries the `queue` / `slot_hashes` / `instructions` accounts (`lib.rs:285-315`). The removal is deliberate (the accounts were dropped, not left dangling), so this reads as a design decision to make screening deposit-only.

The public documentation still states otherwise. The Compliance page says, verbatim: **"Arcane integrates the Range Risk and Compliance API to screen every deposit and withdrawal in real time, before the protocol touches it,"** while the Home page separately says **"every deposit."** Because the on-chain program is permissionless, a caller that bypasses the front end can withdraw to an address the docs imply is screened.

Impact

Compliance-posture and documentation-consistency only; no fund loss to other users. A direct program caller (or any flow where the front end is omitted) can withdraw to a sanctioned/high-risk recipient the protocol claims to screen. The exposure is to the protocol's own compliance guarantee, not to pooled funds.

Recommendation

Confirm the intended model. Either (a) restore the on-chain `verify_risk_score_feed(&recipient)` (and the `queue` / `slot_hashes` / `instructions` accounts) on both withdraw handlers, or (b) if withdrawal screening is deliberately deposit-only, align the Compliance documentation to match - scope on-chain enforcement to deposits and describe any withdrawal screening as a front-end control a permissionless direct caller can bypass. Also reconcile the inconsistency between the public pages themselves ("every deposit" on the Home page vs "every deposit and withdrawal" on the Compliance page).

Status

Acknowledged

[Q-03] arcane#TokenWithdraw - Destination token accounts are not auto-created on withdrawal

Description

Token withdrawals send funds to the recipient, relayer, and fee-wallet associated token accounts but do not create them. If one of these accounts does not already exist, the withdrawal transaction fails. This is expected Solana behaviour that the caller normally handles, so it is informational.

Vulnerability Details

`recipient_ata` and `relayer_ata` carry `associated_token` constraints but are not declared `init_if_needed`, and each fee-wallet ATA is a plain `transfer_checked` target taken from `remaining_accounts`. If any of these accounts does not exist, the transaction reverts - for `recipient_ata` / `relayer_ata` at Anchor's account validation, and for a fee-wallet ATA at the `transfer_checked` CPI.

Impact

No security impact and no fund loss (the treasury is untouched on revert). The standard Solana pattern is for the caller/relayer to prepend a `CreateIdempotent` instruction in the same transaction, which creates any missing account atomically and cannot be front-run. The only downside is a failed transaction if the client omits it.

Recommendation

For a smoother flow, declare `recipient_ata` (and `relayer_ata`) as `init_if_needed` with the relayer as payer - the feature is already enabled elsewhere in the program - or document that clients must prepend `CreateIdempotent` for all destination accounts. Optionally, accrue a missing fee-wallet's share to the relayer instead of reverting.

Status

Resolved

Centralisation

The Arcane project values security and utility over decentralisation.

The owner executable functions within the protocol increase security and functionality but depend highly on internal team responsibility.



Centralised

Decentralised

Conclusion

After Hashlock's analysis, the Arcane project seems to have a sound and well-tested code base, now that our vulnerability findings have been resolved and acknowledged. Overall, most of the code is correctly ordered and follows industry best practices. The code is well commented on as well. To the best of our ability, Hashlock is not able to identify any further vulnerabilities.

Our Methodology

Hashlock strives to maintain a transparent working process and to make our audits a collaborative effort. The objective of our security audits is to improve the quality of systems and upcoming projects we review and to aim for sufficient remediation to help protect users and project leaders. Below is the methodology we use in our security audit process.

Manual Code Review:

In manually analysing all of the code, we seek to find any potential issues with code logic, error handling, protocol and header parsing, cryptographic errors, and random number generators. We also watch for areas where more defensive programming could reduce the risk of future mistakes and speed up future audits. Although our primary focus is on the in-scope code, we examine dependency code and behaviour when it is relevant to a particular line of investigation.

Vulnerability Analysis:

Our methodologies include manual code analysis, user interface interaction, and white box penetration testing. We consider the project's website, specifications, and whitepaper (if available) to attain a high-level understanding of what functionality the smart contract under review contains. We then communicate with the developers and founders to gain insight into their vision for the project. We install and deploy the relevant software, exploring the user interactions and roles. While we do this, we brainstorm threat models and attack surfaces. We read design documentation, review other audit results, search for similar projects, examine source code dependencies, skim open issue tickets, and generally investigate details other than the implementation.

Documenting Results:

We undergo a robust, transparent process for analysing potential security vulnerabilities and seeing them through to successful remediation. When a potential issue is discovered, we immediately create an issue entry for it in this document, even though we have not yet verified the feasibility and impact of the issue. This process is vast because we document our suspicions early even if they are later shown to not represent exploitable vulnerabilities. We generally follow a process of first documenting the suspicion with unresolved questions, and then confirming the issue through code analysis, live experimentation, or automated tests. Code analysis is the most tentative, and we strive to provide test code, log captures, or screenshots demonstrating our confirmation. After this, we analyse the feasibility of an attack in a live system.

Suggested Solutions:

We search for immediate mitigations that live deployments can take and finally, we suggest the requirements for remediation engineering for future releases. The mitigation and remediation recommendations should be scrutinised by the developers and deployment engineers, and successful mitigation and remediation is an ongoing collaborative process after we deliver our report, and before the contract details are made public.

Disclaimers

Hashlock's Disclaimer

Hashlock's team has analysed these smart contracts in accordance with the best industry practices at the date of this report, in relation to: cybersecurity vulnerabilities and issues in the smart contract source code, the details of which are disclosed in this report, (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

Due to the fact that the total number of test cases is unlimited, the audit makes no statements or warranties on the security of the code. It also cannot be considered as a sufficient assessment regarding the utility and safety of the code, bug-free status, or any other statements of the contract. While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only. We also suggest conducting a bug bounty program to confirm the high level of security of this smart contract.

Hashlock is not responsible for the safety of any funds and is not in any way liable for the security of the project.

Technical Disclaimer

Smart contracts are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the smart contract can have their own vulnerabilities that can lead to attacks. Thus, the audit can't guarantee the explicit security of the audited smart contracts.

About Hashlock

Hashlock is an Australian-based company aiming to help facilitate the successful widespread adoption of distributed ledger technology. Our key services all have a focus on security, as well as projects that focus on streamlined adoption in the business sector.

Hashlock is excited to continue to grow its partnerships with developers and other web3-oriented companies to collaborate on secure innovation, helping businesses and decentralised entities alike.

Website: hashlock.com.au

Contact: info@hashlock.com.au



#hashlock.